UNIT IV

Intel 8086 Microprocessor: Introduction-Intel 8086 Hardware – Pin description – External memory Addressing – Bus cycles – Interrupt Processing. Addressing modes - Instruction set – Assembler Directives.

1. What are the difference between 8085 & 8086?***

8085	8086
8-bit microprocessor	16-bit microprocessor
2^{16} memory locations	2^{20} memory locations
Sequential facility	Pipelined architecture available
Low speed	High speed
It does not have much operational instructions when compared to 8086	It allows to have much large set of operation and instruction.

2. What are the functional units available in 8086?

The internal functions of the 8086 processor are portioned logically into two processing units.

1. Bus Interface Unit (BIU)
2. Execution unit (EU)
   - BIU & EU function independently.

The **BIU** contains	The **EU** contains
1.Segment registers 2.Instruction pointer 3. Instruction queue.	1.ALU 2.General purpose registers 3. Index registers 4. Pointers 5.Flag register

3. What **are the functions of BIU and EU?***

- The BIU and EU function independently.

Bus Interface Unit (BIU):

- The BIU interfaces the 8086 to the outside of the world.
- The BIU fetches instruction, reads data from memory and ports and write data to the memory and I/O ports.

Execution unit (EU):

- EU receives program instruction codes and data from the BIU, executes these instructions and stores the result in the general registers or output them through the BIU.

4. How the instructions are classified in 8086?**

Instructions of 8086 are classified into **six** groups.

1. Data transfer instructions
2. Arithmetic instructions
3. Bit manipulation instructions
4. String instructions
5. Program execution transfer instructions
6. Processor control instructions

5. What are the types of addressing modes in 8086?***

The method of specifying the data to be operated by the instruction is called **Addressing.**

The 8086 has **12** addressing modes:

1. Register Addressing Mode
2. Immediate Addressing Mode]
3. Direct Addressing Mode
4. Register Indirect Addressing Mode
5. Based Addressing Mode
6. Indexed Addressing Mode
7. Based Indexed Addressing Mode
8. String Addressing Mode
9. Direct I/O port Addressing Mode
10. Indirect I/O port Addressing Mode
11. Relative Addressing Mode
12. Implied Addressing Mode.

6. Define segment register? List types of segment in 8086 memory.***

Segment registers are in Bus Interface Unit (BIU) of 8086.

Most of the registers contain data/instruction offset within 64KB memory segment.

There are **four** different 64 KB segments for instructions, stack, data & extra data.

The segment registers are:

1. **C**ode **S**egment(CS)
2. **S**tack **S**egment(SS)
3. **D**ata **S**egment(DS)
4. **E**xtra **S**egment(ES)

7. What is 8086 directives?

An assembler is a program which translates an assembly language program into machine language program.
An assembly language program consists of two types of statements:
Instruction & Directives.
The instructions are translated to machine codes by the assembler, whereas the directives are not translated to machine codes.
Some assembler directives are:

1. Borland Turbo Assembler (TASM)
2. IBM Macro Assembler (MASM)
3. Intel 8086 Macro Assembler (ASM)
4. Microsoft Macro Assembler.

8. What is the need of assembler directives?**

- An assembler directive is a statement to give direction to the assembler to perform the task of assembly process.
- The assembler directives control organization ofthe program and provide necessary information to the assembler to understand assembly language program to generate machine codes.
- They indicate how an operand or a section of a program is to be processed by the assembler.
- An assembler supports directives to define data, to organize segments, to control procedures, to define macros etc.

9. Give examples for some assembler directives that are specific to 8086 assembly language.***

The general assembler directives are: ASSUME , EXTRN, GROUP, INCLUDE, LABEL, MACRO, ORG, PTR, PROC, PUBLIC, RECORD, SEGMENT, STRUC, EVEN, EQU, END, ENDM, ENDS, ENDP, DT, DQ , DD,DW, DB.

10. Explain ASSUME.

The ASSUME directive enables error-checking for register values.

It is used to inform the assembler the names of the logical segments, which are to be assigned to the different segments used in an assembly language program.

Format:
ASSUMEsegregister:name[[,segregister:name]]…
ASSUME dataregister:type[[,dataregister:type]]…
ASSUME register:**ERROR**[[,register**:ERROR**]]…
ASSUME [[register:]] **NOTHING** [[,register:**NOTHING**]]…

After an ASSUME is put into effect, the assembler watches for changes to the values of the given registers. ERROR generates an error if the register is used. NOTHING removes register error-checking.

Examples:

```
ASSUME CS : CODE
ASSUME DS : DATA
```

11. Explain SEGMENT.**

SEGMENT is used to indicate the start of a logical segment. It defines a program segment called *name* having segment attributes *align* (BYTE, WORD, DWORD), *combine* (PUBLIC, STACK), *use* (USE16, USE32, FLAT), and *class.*
The ENDS statement indicates the end of the program.

Format:

name **SEGMENT** *type (WORD or PUBLIC)*
statements
name **ENDS**

Examples:

```
CODE    SEGMENT    WORD
        .
        .
        .
CODE    ENDS
```

12. Explain EVEN.

EVEN (Align on Even memory Address):

The EVEN directive tells the assembler to increment the location counter to the next even address if it is not already at an even address.

Format:

```
        EVEN
```

Examples:

```
SALES           DB
EVEN
DATA_ARRAY      DW    100 DUP (?)
```

13. Explain DD, DQ and DT.***

DD (Define Double Word):

It can be used to define data like **DWORD** (4bytes)

Format:

Name of the variable DD *Initial values*

Example:

```
NUMBER          DD    12345678
```

DQ (Define Quad Word):
It can be used to define data like **QWORD** (8 bytes).
Format:
Name of the variable DQ *Initial values*
Example:
TABLE DQ 1234567812345678
DT (Define Ten Bytes):
It can be used to define data like **TBYTE** (10 bytes).
Format:
Name of the variable DT *Initial values*
Example:
AMOUNT DT 12345678123456781234

14. What is the role of TF and IF flags in 8086?***

TF (Trap Flag)
Setting TF puts the 8086 in the single step mode. In this mode, the 8086 generates an internal interrupt after the execution of each instruction.
IF (Interrupt Flag)
Setting IF causes the 8086 to receive external maskable interrupts through INTR pin. Clearing IF disable these interrupts.

15. What is the function of T and D flags in 8086?

D Flag- String Direction Flag:
It is used to set direction in string operation.
T Flag- Single Step Trap Flag:
It is used for single stepping through a program.

16. What is meant by software interrupt in 8086?

The software interrupt are the program instructions. These instructions are inserted at desired locations in a program. While running a program, if a software interrupt is encountered then the processor executes an interrupt service routine (ISR).

17. Explain the instructions AAA ,AAS,AAM & AAD?***

AAA:
ASCII Adjust for Addition instruction adjusts the binary result of ADD or ADC instruction.If bits 0-3 of AL contain a value greater than 9, or if the Auxiliary carry flag is set, the CPU adds 06 to AL and adds 1 to AH. The bits 4-7 are set to zero.
(AL) ← (AL) + 6
(AH) ← (AH) + 1

(AF) ← 1

Example:

AAA

Before execution

00	0B
AH	AL

After execution

01	01
AH	AL

AAS:

ASCII Adjust for Subtraction instruction adjusts the binary result of a SUB or SBB instruction.

If D_3-D_0 OF AL > 9

(AL) ← (AL) - 6

(AH) ← (AH) - 1

(AF) ← 1

AAM:

ASCII Adjust for Multiplication instruction adjusts the binary results of a MUI instruction. AL is divided by 10($0A_H$) and the quotient is stored in AH. The remainder is stored in AL.

(AH) ← (AL/$0A_H$)

(AL) ← Remainder

AAD:

ASCII Adjust for Division instruction adjusts unpacked BCD dividend in AX before a division operation. AH is multiplied 10($0A_H$) and added to AL. AH is set to zero.

(AL) ← (AH X $0A_H$) + (AL)

(AH) ← 0

18. What is Intra-segment (NEARJMP) Inter segment (FARJMP)?***

A **NEAR-JMP (Intra segment)** is a jump where destination location is in the same code segment. In this case only IP (**I**nstruction **P**ointer) is changed.

IP = IP + signed displacement

A **FAR-JMP (Inter-segment)** is a jump where destination location is from a different segment. In this case both IP (**I**nstruction **P**ointer) and CS (**C**ode **S**egment) are changes as specified in the destination.

19. What is NEAR-CALL&FAR-CALL?

Intra-Segment CALL:

A NEAR-CALL is a call to a procedure which is in the same code segment as the CALL instruction. When 8086 executes a NEAR-CALL instruction, it decrements the stack pointer (SP) by 2 and copies the offset of the next instruction after the CALL on

the stack. It loads IP with the offset of the first instruction of the procedure in same segment.

Inter-Segment CALL:

A FAR-CALL is a call to a procedure which is in a different segment from that which contains the CALL instruction. When 8086 executes a FAR-CALL, it decrements the SP by 2 and copies the content of the CS register to the stack. It then decrements SP by 2 again and copies the offset of the instruction after the CALL to the stack. Finally it loads CS with the segment base of the segment which contains the procedure and IP with the offset of the first instruction of the procedure in that segment.

20. How 8086 is organized to facilitate memory read/write operation for addressing external memory?***

The 16 bit members of the 8086 family can load a word from any arbitrary address. The processor fetches the lower order byte of the value from the address specified and the higher order byte from the next consecutive address.

The memory bank is selected by $\overline{BHE}$ and A_0.

The EVEN memory bank is selected by the address line A_0.

The ODD memory bank is selected by the control signal $\overline{BHE}$.

Any memory location in the memory bank is selected by the address line A_1 TO A_{19}.

$\overline{BHE}$	A_0	Characteristics
0	0	Whole word
0	1	Upper byte from/to ODD address
1	0	Lower byte from/to EVEN address
1	1	None

Program, data and stack memories occupy the same space. The total addressable memory size is 1 MB. As the most of the processor instructions use 16-bit pointers, the processor can effectively address only 64KB of memory. To access memory outside of 64 KB the CPU uses special segment registers to specify where the code, stack and data 64 KB segments are positioned within 1 MB of memory.

21. Write about interrupt priority of 8086.

Interrupt	Priority
INT n, INTO, Divide Error	Highest
NMI	↓
INTR	↓
Single step	lowest

Table: the priority of interrupts of 8086.

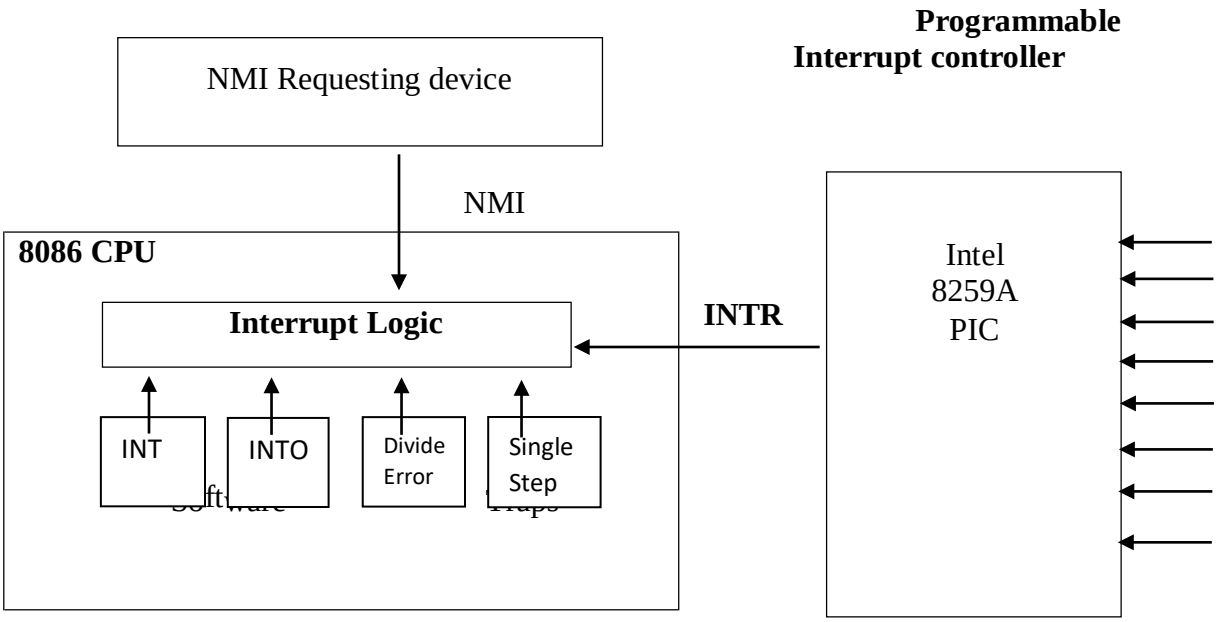
The software interrupts except Single Step interrupt have the highest priority; followed by NMI, followed by INTR. Single step interrupt has the least priority. The 8086 checks for internal interrupts before for any hardware interrupt. Therefore software interrupts have higher priority than hardware interrupt.

22. What are the interrupts in 8086***

1. Hardware Interrupt – External Uses INTR and NMI
2. Software Interrupt – Internal – from INT or INTO
3. Processor Interrupt – Traps and 10 Software Interrupts

External –generated outside the CPU by other hardware.
(INTR, NMI)

Internal –generated within CPU as of an instruction or operation.
(INT, INTO, Divide Error and Single Step)



8086 INTERRUPT CONNECTIONS

23. How the physical address for fetching the next instruction to be executed is obtained in 8086?

The physical address is obtained by appending four zeros to the content present in CS register and then adding the content of IP register with the above value.

24. What is pipelining?***

The instruction queue is a First-In-First-Out (FIFO) group of registers where 6 bytes of instruction code is pre-fetched from memory ahead of time. It is being done to speed up program execution by overlapping instruction fetch and execution. This mechanism is known as **PIPELINING.**

- Fetching the next instruction while the current instruction executes is called **PIPELINING.**

Unit IV

Intel 8086 Microprocessor: Introduction-Intel 8086 Hardware – Pin description – External memory Addressing – Bus cycles – Interrupt Processing. Addressing modes - Instruction set – Assembler Directives.

INTEL 8086 MICROPROCESSOR

1. ARCHITECTURE OF 8086:

The internal architecture of 8086 is divided into two separate units. They are

(i) Bus Interface Unit (BIU)

(ii) Execution Unit (EU)

The two units function independently.

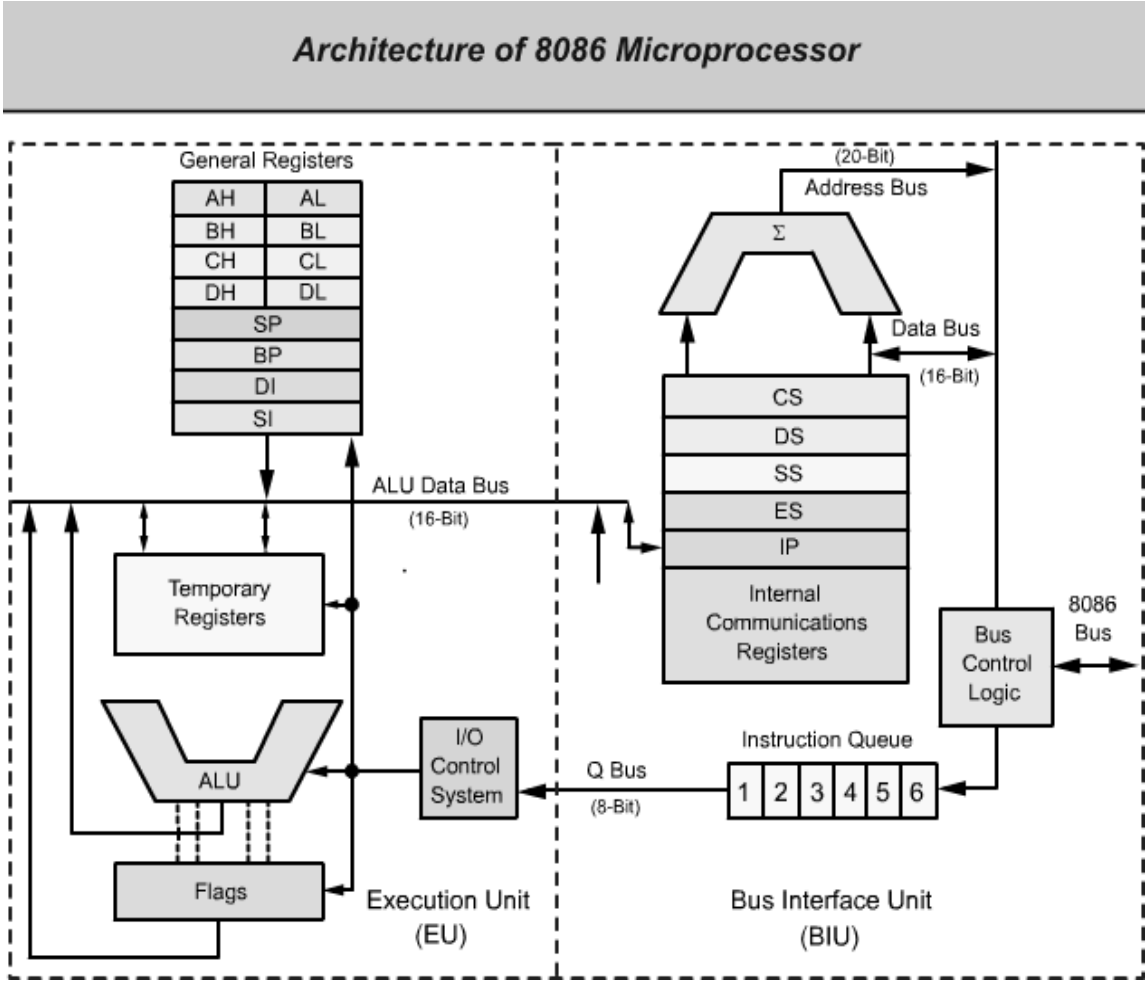
The BIU is needed to fetch instruction, read operands, and write results. The execution unit is used to execute instructions that have already been fetched by the BIU.

BUS INTERFACE UNIT

- The BIU is used to handle all transfers of data and addresses on the buses for the Execution unit.
- The BIU is used to send addresses, fetch instructions from the memory, read data from ports and memory and write data to ports and memory.
- The BIU contains segment registers, instruction pointer and the instruction queue.

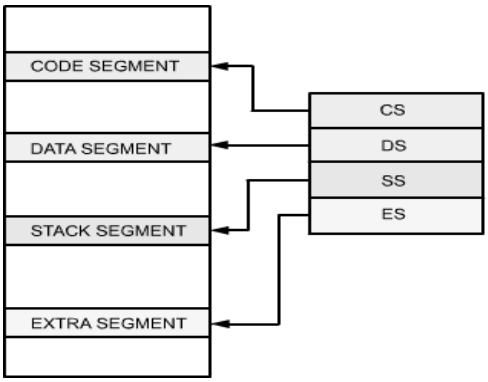
QUEUE

- The queue is a First – In – First – Out (FIFO) group of registers.
- 6 bytes of instruction code are fetched in advance and stored in a queue, while the EU is not using the buses.
- EU fetched instructions from this queue to execute.
- When the EU is ready for the execution of the next instruction, it reads the byte from the instruction queue.
- The time required to access the memory. So it increases the overall processing is called as “Pipelining”.



SEGMENT REGISTER

- In 8086 memory (1 MB) is divided into number of segments.
- The size of the each segment is 64K bytes.
- A segment is an area that begins with a paragraph boundary, that is, at any location divisible by 16.
- A segment may be located anywhere in the memory.
- Each of these segments can be used for a specific function.
- Code segment is used for storing the instructions.
- The stack segment is used as a stack and it is used to store the return addresses.
- The data and extra segment are used for storing data byte.
- In the assembly language programming, more than one data/code/ stack segments can be defined.
- But only one segment of each type can be accessed at any time.
- The Figure shows different segment and segment registers.



Segments and Segment Register

The BIU contains four segments registers: They are

(i) Code segment register

(ii) Stack segment register

(iii)Data segment register and

(iv) Extra segment register

- These registers are used to hold the upper 16-bits of the starting address of the logical group of memory, called the segment.
- The address of the memory bytes that need to be accessed is generated with the help of address contained in the segment registers and other registers.

CS REGISTER:

- This register contains the initial address of the code segment (CS).
- This address plus the offset value contained in the instruction pointer (IP) indicates the address of an instruction to be fetched for execution.

SS REGISTER:

- The stack segment (SS) register contains the initial address of the stack segment. This address plus the value contained in the stack pointer (SP) is used for stack operations.

DS REGISTER:

- The data segment (DS) register contains the initial address of the current data segment.
- This address plus the offset value in instruction causes a reference to a specific location in the data segment.

ES REGISTER:

- Extra segment is used by some string operations.

- The Extra segment register contains the initial address of the extra segment.
- String instructions always use the ES and DI registers to calculate the physical address for the destination.

ADVANTAGES OF SEGMENT REGISTERS:

1. The segment registers permit a program and its data to be placed in different areas of memory each time the program is executed.

2. The segment registers facilitate the use of separate memory areas for instructions, its data, and the stack.

3. The segment registers are used to allow the instruction, data, or stack portion of a program to be more than 64 K bytes long. The above can be achieved by using more than one code, data, or stack segment.

4. The segment registers are used to allow the memory capacity to be 1MB even though the address associated with the individual instructions are only 16-bits.

INSTRUCTION POINTER (IP):

- The instruction pointer register contains a 16-bit offset address of the instruction that is to be executed next.
- The value contained in the instruction pointer is called as an offset because this value must be added to the base address of the code segment, which is available in the CS register to find the 20-bit physical address.
- The value of the instruction pointer is incremented after executing every instruction.

EXECUTION UNIT (EU):

- The Execution Unit is responsible for executing the instructions.
- The EU decodes and executes instructions.
- BIU provide instructions and data to the EU.

The execution unit contains the following sections:

- Control circuitry, Instruction decoder, ALU and nine 16-bit registers.
- The nine registers are AX, BX, CX, DX, SP, BP, SI and DI and flag register.

GENRAL PURPOSE REGISTERS:

General Purpose Registers:

		15 8	7 0
Accumulator Register	AX	AH	AL
Base Register	BX	BH	BL
Count Register	CX	CH	CL
Data Register	DX	DH	DL

The 8086 has four 16-bit general purpose registers: AX, BX, CX and DX. The above 16-bit registers can also be used as 8-bit registers. This BH, BL, CH, CL, DH, DL.

AX REGISTER:

- The AX register is also called as "Accumulator".
- The use of accumulator registers is assumed by some instructions like divide, rotate, shift etc.
- In such kind of instructions, the user loads the accumulator properly before executing the instruction.
- In this case, the complete 16-bit, or only its lower 8-bit AL is used.

BX REGISTER:

- The BX register is called as "Base Register".
- The contents of this register can be used to address the memory.
- All memory references use the content of this register for addressing by using the DS as the default segment register.

CX REGISTER:

- The CX register is called as "Count Register".
- Some instructions like SHIFT, ROTATE and LOOP use the contents of CX as a counter.
- CX register contains the number of times the loop is to be executed.

DX REGISTER:

- The DX register is known as "Data Register".
- Some input/output operations require the use of this register.
- The DX register is used to hold the high 16-bit result in 16 x 16 multiplications or the high 16-bit dividend in 32/16 division and the 16-bit remainder after the division.

STACK POINTER REGISTER:

- A stack is a block of memory to store address or data.
- The base address of the stack is stored in the Stack segment register.
- The special register called the stack pointer contains the address of the top of the stack.
- The stack pointer contains the offset of the data that has been stored latest on the stack.

BASE POINTER REGISTER:

- Base pointer register is also used to access the data from the stack.
- The value in Sp always represents the offset of the top of the stack.
- BP register also contains an offset relative to SS register.

- With the help of BP register, it is possible to access any location within the stack segment of the memory.

INDEX REGISTER:

- The 8086 contain two index registers Source Index (SI) register and Destination Index (DI) register.
- The main use of three register is to hold the offset of a data in one segments.

CONTROL CIRCUITARY, INSTRUCTION DECODER AND ALU:

- The Execution Unit fetches instruction from the instruction queue.
- The above instruction is stored in the decoder.
- The decoder translates each instruction into sequence of actions, which the EU carries out.
- The ALU is used to perform the arithmetic and logic operations.
- All the above actions are controlled by the control circuitry.
- Control circuitry generates appropriate signals at fixed intervals of time.

8086 FLAG REGISTER:

A flag register is a 16-bit register. A flag is a flip-flop. The flag register contains nine active flags. Out of the above nine flags, six flags are called status flags or conditional flags and the remaining three flags are called control flags.

The six conditional flags are:

1. The carry Flag (CF)
2. The Auxiliary Carry Flag (AF)
3. The Zero Flag (ZF)
4. The Overflow Flag (OF)
5. The Sign Flag (SF)
6. The Parity Flag (PF)

The status Flags are used to indicate some condition produced by an instruction. The Execution Unit sets or resets these flags at the completion of execution of the arithmetic or logical instruction.

Conditional Flags

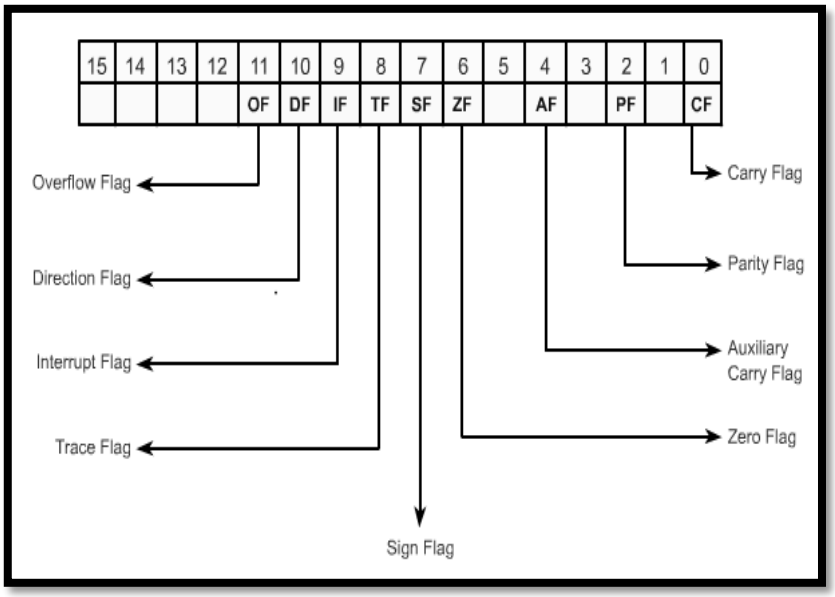
Flag	Name	Function
CF	*Carry Flag*	=1 if high-order bit carry/borrow =0 otherwise
PF	*Parity Flag*	=1 if low order 8-bits of result contain even parity =0 otherwise
AF	*Auxiliary Carry Flag*	=1 if carry from / borrow to lower nibble of AL =0 otherwise
ZF	*Zero Flag*	=1 if result is zero =0 otherwise
SF	*Sign Flag*	=1 if MSB of result is 1 (-ve sign) =0 if MSB of result is 0 (+ve sign)
OF	*Overflow Flag*	=1 if result is out of range =0 otherwise

The three control flags are:

1. The Directory Flag (DF)
2. The Trap Flag (TF)
3. The Interrupt Flag (IF)

These flags are used to control certain operations of the processor.

The control flags are set or reset by the specific instruction. The following figure shows the format of the 8086 flag register. The first row indicates the bit positions and the second row indicates the corresponding flags.



8086 flag Register

Status Flags or Conditioal Flags:

CARRY FLAG (CF)

CF is set if there is a carry or borrow for the most significant bit from addition or subtraction operation respectively.

PARITY FALG (PF)

PF is set, if the result contains an even number of 1's. PF is reset, if the result conatins an odd number of 1's.

AUXILIARY CARRY FLAG (AF)

AF is set if there is a carry from the low nibble into the high nibble during addition or a borrow from the high nibble into the low nibble during subtraction of the low order 8-bit of a 16-bit number. Otherwise, Af is reset.

ZERO FLAG (ZF)

ZF is set, if the result of the arithmetic operation is zero. Otherwise it is reset.

SIGN FLAG (SF)

SF is set if the most significant bit of the result is one; otherwise, it is zero. The value of SF =0 indicates that the sign of the result is positive. If SF =1, the result is negative number.

CONTROL FLAGS

TRAP FLAG (TF)

If TF is set, the 8086 works in the single step mode. In the single step mode, one instruction is executed at a time. This type of operation is very useful for debugging programs.

INTERRUPT FLAG (IF)

If IF is set to 1, the processor recognizes all the interrupts; if IF is cleared to zero, the processor ignores interrupts that come from the external devices.

DIRECTION FLAG (DF)

DF is used in string processing. When Df is set to 1, the string is processed backward. If DF is cleared to zero, the string is processed forward.

OVERFLOW FLAG (OF)

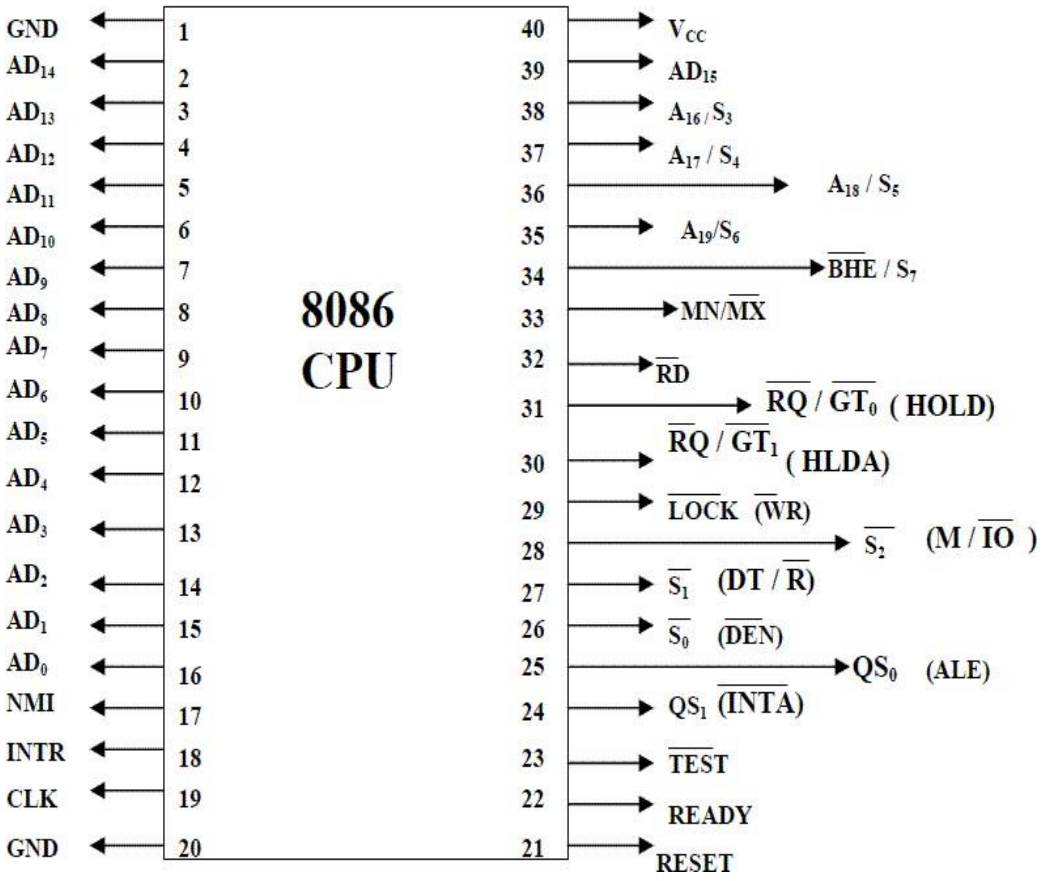
OF is set if there is an arithmetic overflow, i.e., if the size of the result exceeds the capacity of the destination location.

2. PIN DIAGRAM AND PIN DESCRIPTION OF 8086:

Figure shows the Pin diagram of 8086. The description follows it.

- The Microprocessor 8086 is a 16-bit CPU available in different clock rates and packaged in a 40 pin CERDIP or plastic package.
- The 8086 operates in single processor or multiprocessor configuration to achieve high performance. The pins serve a particular function in minimum mode (single processor mode) and other function in maximum mode configuration (multiprocessor mode).

Pin Diagram of 8086



- The Microprocessor 8086 is a 16-bit CPU available in different clock rates and packaged in a 40 pin CERDIP or plastic package.
- The 8086 operates in single processor or multiprocessor configuration to achieve high performance. The pins serve a particular function in minimum mode (single processor mode) and other function in maximum mode configuration (multiprocessor mode).
- The 8086 signals can be categorized in three groups.
    - The first are the signal having common functions in minimum as well as maximum mode.
    - The second are the signals which have special functions for minimum mode
    - The third are the signals having special functions for maximum mode.

- The following signal descriptions are common for both modes.
- AD15-AD0: These are the time multiplexed memory I/O address and data lines.
    - Address remains on the lines during T1 state, while the data is available on the data bus during T2, T3, Tw and T4. These lines are active high and float to a tristate during interrupt acknowledge and local bus hold acknowledge cycles.
- A19/S6, A18/S5, A17/S4, A16/S3 : These are the time multiplexed address and status lines.
    - During T1 these are the most significant address lines for memory operations.
    - During I/O operations, these lines are low.
    - During memory or I/O operations, status information is available on those lines for T2, T3, Tw and T4.
    - The status of the interrupt enable flag bit is updated at the beginning of each clock cycle.
    - The S4 and S3 combinely indicate which segment register is presently being used for memory accesses as in below fig.
    - These lines float to tri-state off during the local bus hold acknowledge. The status line S6 is always low.
    - The address bits are separated from the status bit using latches controlled by the ALE signal.

S4	S3	Indication
0	0	Alternate Data
0	1	Stack
1	0	Code or None
1	1	Data
0	0	Whole word
0	1	Upper byte from or to even address
1	0	Lower byte from or to even address

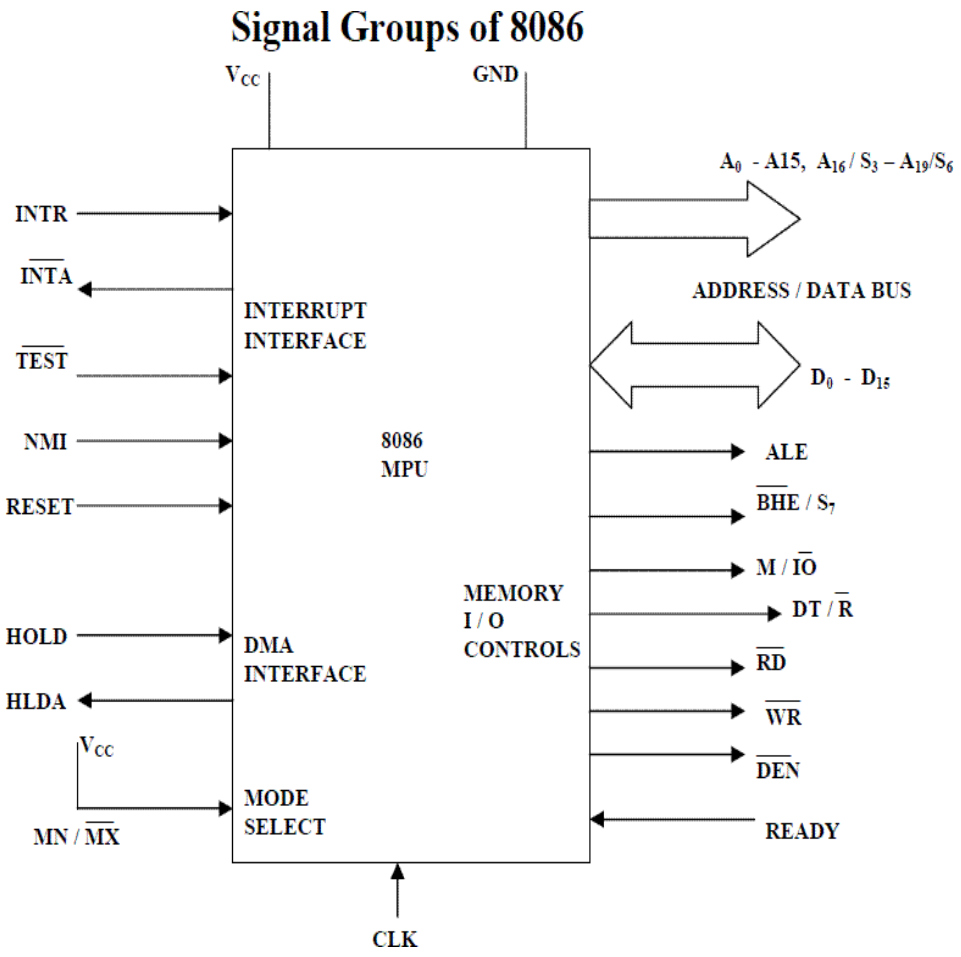
- **BHE/S7:** The bus high enable is used to indicate the transfer of data over the higher order (D15-D8) data bus as shown in table. It goes low for the data transfer over D15-D8 and is used to derive chip selects of odd address memory bank or peripherals. BHE is low during T1 for read, write and interrupt acknowledge cycles, whenever a byte is to be transferred on higher byte of data bus. The status information is available during T2, T3 and T4. The signal is active low and tristated during hold. It is low during T1 for the first pulses of the interrupt acknowledge cycle.
- **RD – Read:** This signal on low indicates the peripheral that the processor is performing memory or I/O read operation. RD is active low and shows the state for T2, T3, Tw of any read cycle. The signal remains tristated during the hold acknowledge.

- **READY**: This is the acknowledgement from the slow device or memory that they have completed the data transfer. The signal made available by the devices is synchronized by the 8284A clock generator to provide ready input to the 8086. the signal is active high.

- **INTR-Interrupt Request:** This is a triggered input. This is sampled during the last clock cycles of each instruction to determine the availability of the request. If any interrupt request is pending, the processor enters the interrupt acknowledge cycle. This can be internally masked by resulting the interrupt enable flag. This signal is active high and internally synchronized.

- **TEST**: This input is examined by a 'WAIT' instruction. If the TEST pin goes low, execution will continue, else the processor remains in an idle state. The input is synchronized internally during each clock cycle on leading edge of clock.

- **CLK- Clock Input:** The clock input provides the basic timing for processor operation and bus control activity. It's an asymmetric square wave with 33% duty cycle.



The following pin functions are for the minimum mode operation of 8086,

- **M/IO – Memory/IO:** This is a status line logically equivalent to S2 in maximum mode. When it is low, it indicates the CPU is having an I/O operation, and when it is high, it indicates that the CPU is having a memory operation. This line becomes active high in the previous T4 and remains active till final T4 of the current cycle. It is tristated during local bus "hold acknowledge ".

- **INTA** – **Interrupt Acknowledge**: This signal is used as a read strobe for interrupt acknowledge cycles. i.e. when it goes low, the processor has accepted the interrupt.

- **ALE – Address Latch Enable:** This output signal indicates the availability of the valid address on the address/data lines, and is connected to latch enable input of latches. This signal is active high and is never tristated.

- **DT/R – Data Transmit/Receive**: This output is used to decide the direction of data flow through the transceivers (bidirectional buffers). When the processor sends out data, this signal is high and when the processor is receiving data, this signal is low.

- **DEN – Data Enable:** This signal indicates the availability of valid data over the address/data lines. It is used to enable the transceivers (bidirectional buffers) to separate the data from the multiplexed address/data signal. It is active from the middle of T2 until the middle of T4. This is tristated during 'hold acknowledge' cycle.

- **HOLD, HLDA- Acknowledge:** When the HOLD line goes high, it indicates to the processor that another master is requesting the bus access. The processor, after receiving the HOLD request, issues the hold acknowledge signal on HLDA pin, in the middle of the next clock cycle after completing the current bus cycle.

- At the same time, the processor floats the local bus and control lines. When the processor detects the HOLD line low, it lowers the HLDA signal. HOLD is an asynchronous input, and is should be externally synchronized. If the DMA request is made while the CPU is performing a memory or I/O cycle, it will release the local bus during T4 provided:

  1. The request occurs on or before T2 state of the current cycle.

  2. The current cycle is not operating over the lower byte of a word.

  3. The current cycle is not the first acknowledge of an interrupt acknowledge sequence.

  4. A Lock instruction is not being executed.

The following pin functions are applicable for maximum mode operation of 8086,

- **S2, S1, S0 – Status Lines:** These are the status lines which reflect the type of operation, being carried out by the processor. These become activity during T4 of the previous cycle and active during T1 and T2 of the current bus cycles.

- **LOCK:** This output pin indicates that other system bus master will be prevented from gaining the system bus, while the LOCK signal is low. The LOCK signal is activated by the 'LOCK' prefix instruction and remains active until the completion of the next instruction. When the CPU is executing a critical instruction which requires the system bus, the LOCK prefix instruction ensures that other processors connected in the system will not gain the control of the bus.

The 8086, while executing the prefixed instruction, asserts the bus lock signal output, which may be connected to an external bus controller. By prefetching the instruction, there is a considerable speeding up in instruction execution in 8086. This is known as instruction pipelining.

S2	S1	S1	INDICATION
0	0	0	Interrupt Acknowledgement
0	0	1	Read I/O port
0	1	0	Write I/O port
0	1	1	Halt
1	0	0	Code Access
1	0	1	Read Memory
1	1	0	Write Memory
1	1	1	Passive

- At the starting the CS:IP is loaded with the required address from which the execution is to be started. Initially, the queue will be empty an the microprocessor starts a fetch operation to bring one byte (the first byte) of instruction code, if the CS:IP address is odd or two bytes at a time, if the CS:IP address is even.

- The first byte is a complete opcode in case of some instruction (one byte opcode instruction) and is a part of opcode, in case of some instructions (two byte opcode instructions), the remaining part of code lie in second byte.

- The second byte is then decoded in continuation with the first byte to decide the instruction length and the number of subsequent bytes to be treated as instruction data. The queue is updated after every byte is read from the queue but the fetch cycle is initiated by BIU only if at least two bytes of the queue are empty and the EU may be concurrently executing the fetched instructions.

- The next byte after the instruction is completed is again the first opcode byte of the next instruction. A similar procedure is repeated till the complete execution of the

program. The fetch operation of the next instruction is overlapped with the execution of the current instruction. As in the architecture, there are two separate units, namely Execution unit and Bus interface unit.

- While the execution unit is busy in executing an instruction, after it is completely decoded, the bus interface unit may be fetching the bytes of the next instruction from memory, depending upon the queue status.

QS1	QS0	INDICATION
0	0	No Operation
0	1	First Byte of the opcode the queue
1	0	Empty Queue
1	1	Subsequent Byte from the Queue

- RQ/GT0, RQ/GT1 – Request/Grant : These pins are used by the other local bus master in maximum mode, to force the processor to release the local bus at the end of the processor current bus cycle.
- Each of the pin is bidirectional with RQ/GT0 having higher priority than RQ/GT1. RQ/GT pins have internal pull-up resistors and may be left unconnected. Request/Grant sequence is as follows:

  1. A pulse of one clock wide from another bus master requests the bus access to 8086.

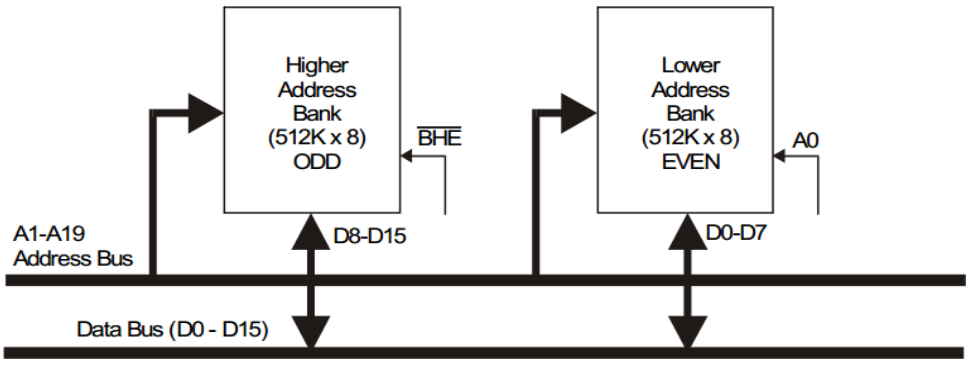
  2. During T4(current) or T1(next) clock cycle, a pulse one clock wide from 8086 to the requesting master, indicates that the 8086 has allowed the local bus to float and that it will enter the 'hold acknowledge' state at next cycle. The CPU bus interface unit is likely to be disconnected from the local bus of the system.

  3. A one clock wide pulse from another master indicates to the 8086 that the hold request is about to end and the 8086 may regain control of the local bus at the next clock cycle. Thus each master to master exchange of the local bus is a sequence of 3 pulses. There must be at least one dead clock cycle after each bus exchange. The request and grant pulses are active low. For the bus request those are received while 8086 is performing memory or I/O cycle, the granting of the bus is governed by the rules as in case of HOLD and HLDA in minimum mode.

3. 8086 EXTERNAL MEMORY ADDRESSING

The 8086 memory address space can be viewed as a sequence of one million bytes in which any byte may contain an 8-bit data element and any two consecutive bytes may contain a 16-bit data element. There is no constraint on byte or word address boundaries. The address space is physically connected to a 16-bit data bus by dividing the address space into two 8-bit banks of up to 512K bytes each.

One bank is connected to the lower half of the 16-bit data bus (D0 – D7) and contains even address bytes. i.e., when A0 bit is low, the bank is selected. The other bank is connected to the upper half of the data bus (D8 - D15) and contains odd address bytes. i.e., when A0 is high and BHE (Bus High Enable) is low, the odd bank is selected. A specific byte within each bank is selected by address lines A1-A19.



Data can be accessed from the memory in four different ways. They are:

- 8 - bit data from Lower (Even) address Bank.
- 8 - bit data from Higher (Od) address Bank.
- 16 - bit data starting from Even Address.
- 16 - bit data starting from Od Address.

4. 8086 INTERRUPT PROCESSING

- The event that causes the interruption is called interrupt.
- The special routine executed to service the interrupt is called interrupt service routine.

Normal program can be interrupted by three ways:

1. By external signal
2. By a special instruction in the program or
3. By the occurrence of some condition

An interrupt caused by an external signal is referred as a hardware interrupt conditional interrupts of interrupts caused by special instructions are called software interrupts.

4.1 8086 INTERRUPT TYPES

- Divide by zero interrupt (type 0)
- Single step interrupt (type 1)

- Non maskable interrupt (type 2)
- Breakpoint interrupt (type 3)
- Overflow interrupt (type 4)

Divide by zero interrupt (type 0)

- When the quotient from either a DIV or IDIV instruction is too large to fit in the result register; 8086 will automatically execute type 0 interrupt.

Single step interrupt (type 1)

- The type 1 interrupt is the single step trap.
- In the single step mode, system will execute one instruction and wait for further direction from user.
- Then user can examine the contents of registers and memory locations and if they are correct, user can tell the system to execute the next instruction.
- This feature is useful for debugging assembly language programs.
- An 8086 system is used in the single step mode by setting the trap flag.
- If the trap flag is set, the 8086 will automatically execute a type 1 interrupt after execution if each instruction.
- But the 8086 has no such instruction to directly set or reset the trap flag.
- These operations can be performed by taking the flag register contents into memory, changing the memory contents so to set or reset trap flag and save the memory contents into flags register.
- To reset the trap flag we have to reset bit 8

Non maskable interrupt (type 2)

- As the name suggests, this interrupt cannot be disabled by any software instruction
- This interrupt is activated by low to high transition on 8086 NMI input pin
- In response 8086 will do a type 2 interrupt

Breakpoint interrupt (type 3)

- Type3 interrupt is used to implement break point function in the system.
- It is produced by execution of the INT3 instruction.
- Break point function is often used as debugging aid in case where single stepping provides more detail than wanted.
- When you insert a break point, the system executes the instruction upto the breakpoint, and then goes to the break point procedure.
- In the breakpoint procedure you can write a program to display register contents, memory contents and other information that is required to debug your program. .
- You can insert as many breakpoints as you want in your program.

Overflow interrupt (type 4):

- It is used to check overflow condition after any signed arithmetic operation in the system.
- For example, if you add the 8-bit signed number 0111 1000(+120 decimal) and the 8 bit signed number 0110 0010 (-98 decimal).
- In signed number, MSB is reserved for sign and other bit represent magnitude of the number.
- In the previous example, after addition of two 8-bit signed numbers result is negative, since it is too large to fit in 7bits.
- To delete this condition in the program you can put interrupt on overflow instruction, INTO, immediately after the arithmetic instruction in the program.
- If the overflow flag is not set when the 8086 executes the INTO instructions, the instruction will simply function as a NOP (no operation).
- However, if the overflow flag is set, indicating an overflow error, the 8086 will executes type4 interrupt after executing the INTO instruction.
- Another way to detect and respond to the overflow error in a program is to put the jump if overflow (JO) instruction immediately after the arithmetic instruction.
- If the overflow flag is set as a result of arithmetic operation, execution will jump to the address specified in the JO instruction.
- At this address you can put an error routine which response in the way you want to overflow.

Software interrupts

Type 0-255:

- The 8025 INT instruction can be used to cause the 8086 to do one of the 256 possible interrupt types.
- The interrupt type is specified by the number as a part of the instructions.
- We can use an INT2 instruction to send execution to an NMI interrupt service routine.
- With the s/w interrupts u can call the desired routines from many different programs in a system.
- The BIOS (Basic Input Output System) routines are called INT instructions. we will summarize interrupt response and how it is serviced by going through following steps.

1. 8086 pushes the flag register on the stack

2. It disables the single step & the INTR input by clearing the trap flag & interrupt flag in the flag register.

3. It saves the current CS &IP register contents by pushing them on the stack.

4. Once these values are loaded in CS & IP, 8086 will fetch the instruction from the new address which is the starting address of ISR

5. An IRET instruction at the end of ISR gets the previous values of CS &IP by popping the CS and IP from the stack.

6. At the end of flag register contents are copied back into flag register by popping the flag register from stack.

MASKABLE INTERRUPT (INTR)

- The 8086 INTR input can be used to interrupt a program execution.
- This interrupt can be enabled or disabled by STI(IF=1)or CLI(if=0)

The 8086 responds to an INTR interrupt as follows:

1. The 8086 has 2 interrupt i) interrupt acknowledge machine cycle the 8086 floats the data bus line AD0-AD15.

2. Once the 8086 receives the interrupt type, it pushes the flag register on the stack, Clears TF &IF, pushes the CS & IP values of the next instruction on the stack.

3. The 8086 then gets the new value of IP from the memory address = 4 times the interrupt type & CS value from memory address = 4 times the interrupt number plus2.

5. ADDRESSING MODES OF 8086

Definition: An instruction acts on any number of operands. The way an instruction accesses its operands is called its **Addressing modes**.

Operands may be of three types:

- Implicit
- Explicit
- Both Implicit and Explicit.

Implicit operands mean that the instruction by definition has some specific operands. The programmers do NOT select these operands.

Example: Implicit operands
XLAT; automatically takes AL and BX as operands AAM; it operates on the contents of AX.

Explicit operands mean the instruction operates on the operands specified by the programmer.

Example: Explicit operands
MOV AX, BX; it takes AX and BX as operands XCHG SI, DI; it takes SI and DI as operands

Implicit and explicit operands

Example: Implicit/Explicit operands
MUL BX; automatically multiply BX explicitly times AX

The location of an operand value in memory space is called the **Effective Address (EA)**

We can classify the addressing modes of 8086 into four groups:

- Immediate addressing
- Register addressing
- Memory addressing
- I/O port addressing

The first three Addressing modes are clearly explained.

Immediate addressing mode & Register addressing mode

Immediate Addressing Mode

In this addressing mode, the operand is stored as part of the instruction. The immediate operand, which is stored along with the instruction, resides in the code segment -- not in the data segment. This addressing mode is also faster to execute an instruction because the operand is read with the instruction from memory. Here are some examples:

Example: Immediate Operands

MOV AL, 20 ; move the constant 20 into register AL
ADD AX, 5 ; add constant 5 to register EAX
MOV DX, offset msg ; move the address of message to register DX

Register addressing mode

In this addressing mode, the operands may be:

- reg16: 16-bit general registers: AX, BX, CX, DX, SI, DI, SP or BP.
- reg8: 8-bit general registers: AH, BH, CH, DH, AL, BL, CL, or DL.
- Sreg: segment registers: CS, DS, ES, or SS. There is an exception: CS cannot be a destination.

For register addressing modes, there is no need to compute the effective address. The operand is in a register and to get the operand there is no memory access involved.

Example: Register Operands

MOV AX, BX ; mov reg16, reg16
ADD AX, SI ; add reg16, reg16
MOV DS, AX ; mov Sreg, reg16

Some rules in register addressing modes:

1. You may not specify CS as the destination operand.

Example: mov CS, 02h –> wrong

2. Only one of the operands can be a segment register. You cannot move data from one segment register to another with a single mov instruction. To copy the value of cs to ds, you would have to use some sequence like:

mov ds,cs -> wrong
mov ax, cs
mov ds, ax -> the way we do it

You should never use the segment registers as data registers to hold arbitrary values. They should only contain segment addresses.

Memory Addressing Modes

Memory (RAM) is the main component of a computer to store temporary data and machine instructions. In a program, programmers many times need to read from and write into memory locations.

There are different forms of memory addressing modes

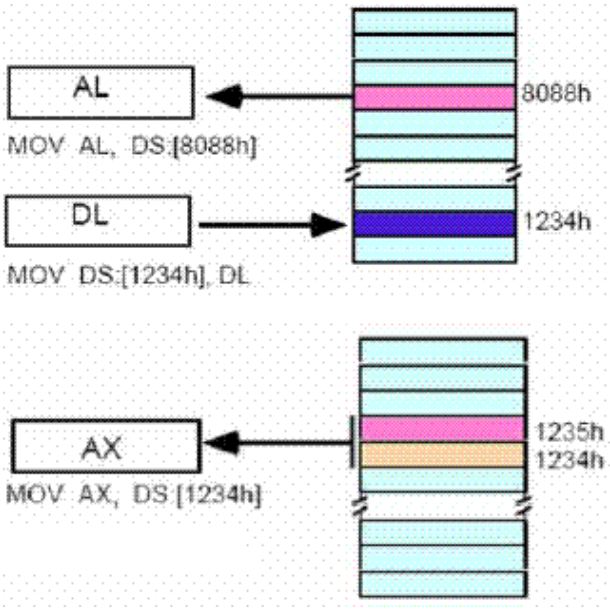
1. Direct Addressing
2. Register indirect addressing
3. Based addressing
4. Indexed addressing

5. Based indexed addressing
6. Based indexed with displacement

Direct Addressing Mode & Register Indirect Addressing Mode

Direct Addressing Mode

The instruction mov al, ds:[8088h] loads the AL register with a copy of the byte at memory location 8088h. Likewise, the instruction mov ds: [1234h],dl stores the value in the dl register to memory location 1234h. By default, all displacement-only values provide offsets into the data segment. If you want to provide an offset into a different segment, you must use a segment override prefix before your address. For example, to access location 1234h in the extra segment (es) you would use an instruction of the form mov ax, es:[1234h]. Likewise, to access this location in the code segment you would use the instruction mov ax, cs: [1234h]. The ds: prefix in the previous examples is not a segment override.



The instruction mov al, ds:[8088h] is same as mov al, [8088h]. If not mentioned DS register is taken by default.

Register Indirect Addressing Mode

The 80x86 CPUs let you access memory indirectly through a register using the register indirect addressing modes. There are four forms of this addressing mode on the 8086, best demonstrated by the following instructions:

mov al, [bx]
mov al, [bp]
mov al, [si]
mov al, [di]

Code Example

MOV BX, 100H
MOV AL, [BX]

The [bx], [si], and [di] modes use the ds segment by default. The [bp] addressing mode uses the stack segment (ss) by default. You can use the segment override prefix symbols if you wish to access data in different segments. The following instructions demonstrate the use of these overrides:

mov al, cs:[bx]
mov al, ds:[bp]
mov al, ss:[si]
mov al, es:[di]

Intel refers to [bx] and [bp] as base addressing modes and bx and bp as base registers (in fact, bp stands for base pointer). Intel refers to the [si] and [di] addressing modes as indexed addressing modes (si stands for source index, di stands for destination index). However, these addressing modes are functionally equivalent. This text will call these forms register indirect modes to be consistent.

Based Addressing Mode and Indexed Addressing Modes

Based Addressing Mode

8-bit or 16-bit instruction operand is added to the contents of a base register (BX or BP), the resulting value is a pointer to location where data resides.

Mov al, [bx],[si]
Mov bl , [bp],[di]
Mov cl , [bp],[di]

Code Example

If bx=1000h
si=0880h
Mov AL, [1000+880]
Mov AL,[1880]

Indexed Addressing Modes

The indexed addressing modes use the following syntax:

mov al, [bx+disp]
mov al, [bp+disp]
mov al, [si+disp]
mov al, [di+disp]

Code Example

```
MOV BX, 100H
MOV AL, [BX + 15]
MOV AL, [BX + 16]
```

If bx contains 1000h, then the instruction mov cl, [bx+20h] will load cl from memory location ds:1020h. Likewise, if bp contains 2020h, mov dh, [bp+1000h] will load dh from location ss:3020. The offsets generated by these addressing modes are the sum of the constant and the specified register. The addressing modes involving bx, si, and di all use the data segment, the [bp+disp] addressing mode uses the stack segment by default. As with the register indirect addressing modes, you can use the segment override prefixes to specify a different segment:

```
mov al, ss:[bx+disp]
mov al, es:[bp+disp]
mov al, cs:[si+disp]
mov al, ss:[di+disp]
```

Based Indexed Addressing Modes & Based Indexed Plus Displacement Addressing Mode

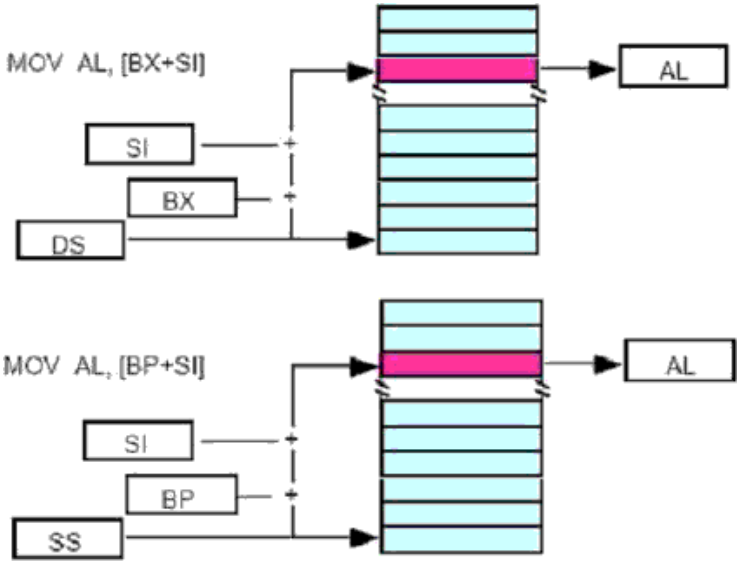
Based Indexed Addressing Modes

The based indexed addressing modes are simply combinations of the register indirect addressing modes. These addressing modes form the offset by adding together a base register (bx or bp) and an index register (si or di). The allowable forms for these addressing modes are:

```
mov al, [bx+si]
mov al, [bx+di]
mov al, [bp+si]
mov al, [bp+di]
```

Code Example

```
MOV BX, 100H
MOV SI, 200H
MOV AL, [BX + SI]
INC BX
INC SI
```



Suppose that bx contains 1000h and si contains 880h. Then the instruction mov al, [bx][si] would load al from location DS:1880h. Likewise, if bp contains 1598h and di contains 1004, mov ax, [bp+di] will load the 16 bits in ax from locations SS: 259C and SS: 259D. The addressing modes that do not involve bp use the data segment by default. Those that have bp as an operand use the stack segment by default.

Based Indexed Plus Displacement Addressing Mode

These addressing modes are a slight modification of the base/indexed addressing modes with the addition of an eight bit or sixteen bit constant. The following are some examples of these addressing modes

mov al, disp[bx][si]
mov al, disp[bx+di]
mov al, [bp+si+disp]
mov al, [bp][di][disp]

Code Example

MOV BX, 100H
MOV SI, 200H
MOV AL, [BX + SI +100H]
INC BX
INC SI

6. INSTRUCTION SETS OF 8086

The instruction set of a processor can defines the basic operations that a programmer can make the device to perform. The 8086 instruction contains no operand, single operand and

two operand instructions. The instruction set will be divided into number of groups of functionally related instructions.

- Data transfer instructions
- Arithmetic instructions
- Bit manipulations instructions
- String manipulations instructions
- Conditional branch instructions
- Unconditional branch instructions
- Iteration control instructions
- Interrupt control instructions
- Processor control instructions

1. Data transfer instructions

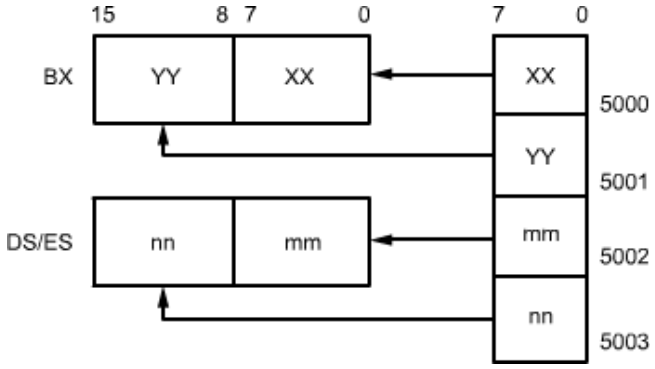
The data transfer instructions transfers' data from one register/memory locations to other register/memory locations. All the store, move, load, exchange, input and output instructions belong to this category. These instructions move single byte or word between a register and I/O ports. Some of the data transfer instructions are listed below.

- MOV d,s
- PUSH d
- POP d
- LEA reg,mem
- LDS reg,mem

The following example will explain the instructions mentioned above

MOV CX,DX copies 16 bit content of DX to CX.

LDS BX, 5000H loads register and DS from memory.



2. Arithmetic instructions

This type of instructions usually perform the arithmetic operations, like Addition, Subtraction, Multiplications, Division, Increment and Decrement operation along with respective ASCII and decimal adjust instructions. The operands are either the registers or memory locations or immediate data depending upon the addressing mode. Some of the arithmetic instructions are given below.

- ADD a,b
- ADC a,b
- AAA
- INC reg/mem
- SUB a,b
- SBB a,b
- CMP a,b
- MUL reg,mem
- DIV reg,mem

The following are the examples for some of the above mentioned instructions.

ADD AX, 0100H Adds to register

SUB 0100H Subtract byte or word, destination is AX

3. Bit manipulation instructions

These types of instructions are used for carrying out the bit by bit shift, rotate in basic logical operations. All the condition code flags are affected depending upon the result. The 8086 provides three groups of bit manipulation instructions. Some of the instructions are listed below.

- AND a, b
- OR a, b
- XOR a, b
- SHL/SAL mem / reg, CNT
- SHR/SAR mem /reg, CNT
- RCL mem/reg, CNT
- RCR mem/reg, CNT

The following are the example for the above mentioned instructions.

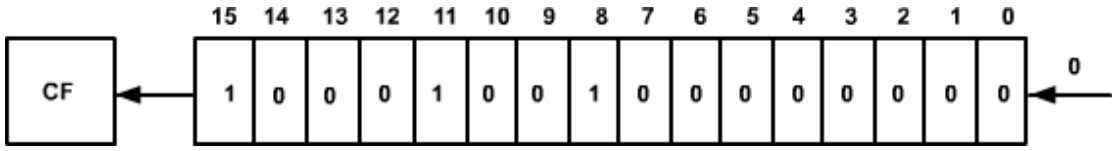
1. AND, Logical AND, Logical OR, Logical Inverter and Logical XOR

The source operand that may be available immediately, register or a memory location ANDed bit by bit to the destination operand that may be a register or a memory location.

Eg: AND AX, 008H

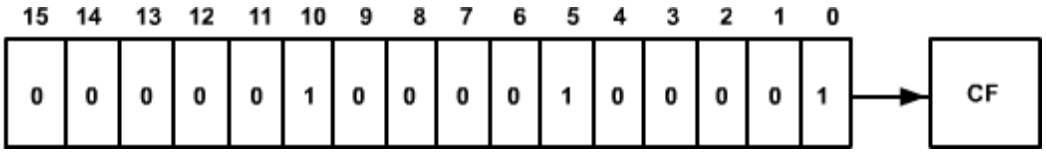
- **SHL/SAL [Shift Logical/Arithmetic Left]**

The instructions shift the operand word or byte bit by bit to the left and insert zeros in the newly introduced least significant bits.



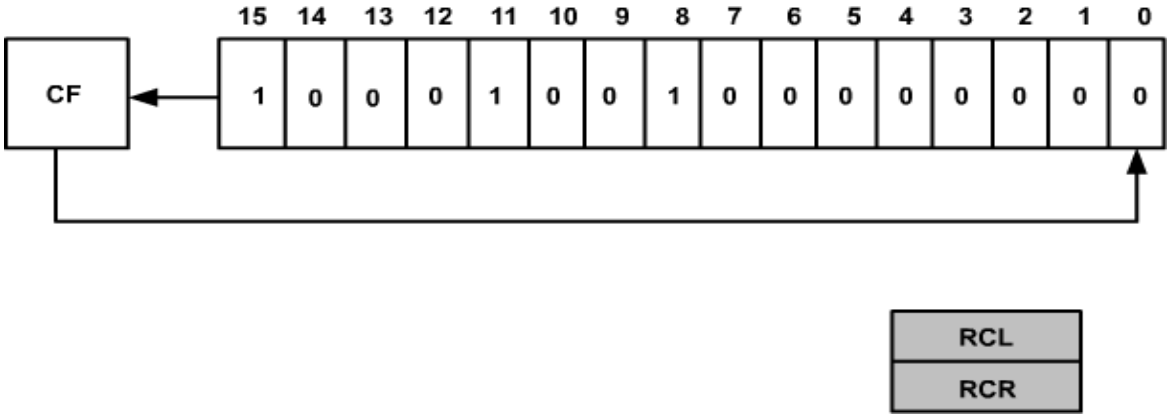
- **SHR/SAR [Shift Logical Right /Arithmetic Right]**

The instructions are same as SHL/SAL; the only difference is shift the operand word or byte bit by bit to the right. The result is stored in the destination operand.



- **RCR/RCR [Rotate Right through Carry/Rotate Left through Carry]**

These instructions rotate the contents (bit-wise) of the destination operand right or left respectively by the specified count through Carry flag (CF). In RCR, the Carry flag is pushed in to the MSB of the operand, and the LSB is pushed into carry flag for each operation. In RCL the carry flag for each operation.



4. String manipulation instructions

String instructions are available to MOVE, COMPARE or SCAS for a value as well as to move string elements to and from the accumulator. A series of the data bytes or words available is memory at consecutive locations, to be referred to collectively or word strings. For referring to a string two parameters are required (a) string or end address of the sting (b) length of the string. Some of the string manipulations are given below.

- MOVSB/MOVSW[Move String Byte or String Word]

These instruction moves 8 or 16 bit data from the memory l0cation addressed by SI to another set of destination location which is addressed by SI.

Eg: MOV AX, 5000H -Source segment address is 5000H

- CMPS [Compare String Byte or String Words]

When two strings of bytes or words are to be compared the CMPs can be used. The length of the strings must be stored in the CX register. If both strings are equal Z=1, other flags are affected in the same way as CMP instruction.

5. Conditional branch instructions

In these instructions, the control is transferred to the specified location provided the result of previous operation satisfies a particular condition, otherwise, the execution continuous in normal flow sequence. All the conditional branch instructions use 8-bit signed displacement these type of instructions do not affect any flag. The typical structure of the conditional branch is follows.

If condition is true,

Then PC←PC + disp 8 otherwise

PC←PC + 2 and execute next instruction.

Some of the conditional branch instructions are given below

- JZ
- JNZ
- JS
- JNS

6. Unconditional branch instructions

These instructions, transfers the execution control to the specified location independent of any status or condition. The CS and IP are unconditionary, modified to the new CS and IP. The 8086 unconditional transfers are

CALL reg/mem/disp 16 ; call subroutine

RET ; return from subroutine

JMP reg/mem/disp 8/disp 16 ; Unconditional jump

- **CALL[Unconditional call]**

This instruction is used to call a subroutine form amain program. While executing this instruction IP is incremented (i.e. address of the next instruction to be executed) and CS on to the stack with the flags and loads the CS and IP register respectively.

- **RET[Return from the Subourtine]**

At the end of the subroutine, the RET must e executed, upon executing the previously stored contents of IP and CS along with flags are retrieved into CS, IP and flag registers from the stack and the main program will be executed the types of procedure and the SP contents.

They are

a) Return with segment

b) Return within segment adding 16-bit immediate displacement to the SP contents

c) Return intersegment

d) Return intersegment adding 16-bit immediate displace ment to the SP contents

- **JMP[Unconditional JumP]**

This instruction transfers the control of execution to the specified address using an 8-bit or 16-bit displacement or CS unconditionally. No flags are affected by this instruction.

7. Iteration control instructions

These instructions execute the part of the program from the label or address specified in the instruction up to the loop instruction, CX number of times. These instructions are given below

- LOOP disp 8 Decrement CX by 1 without affecting flags and loop if CX≠0.
- LOOP E/LOOP Z disp8 Decrement CX by 1 without affecting flags and loop if CX≠0 /not equal .
- JCX Z disp 8 JMP if register CX = 0

8. Interrupt instructions

In the 8086, there are 256 interrupts are defined corresponding to the types from 00Hto FFH. Where an Interrupt instruction executed, the TYPE byte N Is multiplied and the contents of IP and CS of the interrupt service routine will be taken from the hexadecimal multiplication

(N*4) as offset address and 0000 as segment address. The interrupt instruction in8086 is listed below.

INT interrupt number (can be 0 - 255_{10})	→	Software interrupt instructions [INT (32_{10} - 255_{10}) (variable to the user)]
INTO	→	Interrupt on over flow the over flow flag (0F) is set.
IRET	→	Interrupt return when an interrupt service routine is to be called, before transferring control to it. The values are IP, CS and flags are retrieved from the stack to continue the execution of the main program.

Process control instructions

Hardware function in the processor chip can be controlled by these instructions.

There are two types

i. Flag manipulation Instruction

ii. Machine control Instruction

First one directly modifies some of the flags of 8086, later control the bus usage and execution.

A processor control instruction available in 8086 is listed below.

Mnemonics	Comments	Types
STC	Set carry CF 1	
CLC	Clear carry CF 0	
CMC	Complementary carry CF CF	
CLD	Clear direction flag	Flag manipulation
STD	Set direction flag	
CLI	Clear interrupt enable flag	
STI	Set interrupt enable flag	
WAIT	Wait for TEST pin to active	
HLT	Halt the processor	
NOP	No operation	Machine Control
ESC Mem	Escape to external processor	Instruction
LOCK	Lock bus during next instruction	

7. ASSEMBLER DIRECTIVES

Assembly languages are low-level languages for programming computers, microprocessors, microcontrollers, and other IC. They implement a symbolic representation of the numeric machine Codes and other constants needed to program a particular CPU architecture. This representation is usually defined by the hardware manufacturer, and is based on abbreviations that help the programmer to remember individual instructions, registers. An assembler directive is a statement to give direction to the assembler to perform task of the assembly process.

It control the organization if the program and provide necessary information to the assembler to understand the assembly language programs to generate necessary machine codes. They indicate how an operand or a section of the program is to be processed by the assembler.

An assembler supports directives to define data, to organise segments to control procedure, to define macros. It consists of two types of statements: instructions and directives. The instructions are translated to the machine code by the assembler whereas directives are not translated to the machine codes.

Assembler directive 8086 microprocessor

(a) The DB directive

(b) The DW directive

(c) The DD directive

(d) The STRUCT (or STRUC) and ENDS directives (counted as one)

(e)The EQU Directive

(f)The COMMENT directive

(g)ASSUME

(h) EXTERN

(i) GLOBAL

(j) SEGMENT

(k)OFFSET

(l) PROC

(m)GROUP

(n) INCLUDE

Data declaration directives:

1. DB - The DB directive is used to declare a BYTE -2-BYTE variable - A BYTE is made up of 8 bits.

Declaration examples:

Byte1 DB 10h

Byte2 DB 255 ; 0FFh, the max. possible for a BYTE

CRLF DB 0Dh, 0Ah, 24h ;Carriage Return, terminator BYTE

2. DW - The DW directive is used to declare a WORD type variable - A WORD occupies 16 bits or (2 BYTE).

Declaration examples:

Word DW 1234h

Word2 DW 65535; 0FFFFh, (the max. possible for a WORD)

3. DD - The DD directive is used to declare a DWORD - A DWORD double word is made up of 32 bits =2 Word's or 4 BYTE.

Declaration examples:

Dword1 DW 12345678h

Dword2 DW 4294967295 ;0FFFFFFFFh.

4. STRUCT and ENDS directives to define a structure template for grouping data items.

(1) The STRUCT directive tells the assembler that a user defined uninitialized data structure follows. The uninitialized data structure consists of a combination of the three supported data types. DB, DW, and DD. The labels serve as zero-based offsets into the structure. The first element's offset for any structure is 0. A structure element is referenced with the base "+" operator before the element's name.

A Structure ends by using the ENDS directive meaning END of Structure.

Syntax:

STRUCT

Structure_element_name element_data_type?

. . .

. . .

. . .

ENDS

(OR)

STRUC

Structure_element_name element_data_type?

. . .

. . .

. . .

ENDS

DECLARATION:

STRUCT

Byte1 DB?

Byte2 DB?

Word1 DW?

Word2 DW?

Dword1DW?

Dword2 DW?

ENDS

Use OF STRUCT:

The STRUCT directive enables us to change the order of items in the structure when, we reform a file header and shuffle the data. Shuffle the data items in the file header and reformat the sequence of data declaration in the STRUCT and off you go. No change in the code we write that processes the file header is necessary unless you inserted an extra data element.

(5) The EQU Directive

The EQU directive is used to give name to some value or symbol. Each time the assembler finds the given names in the program, it will replace the name with the value or a symbol. The value can be in the range 0 through 65535 and it can be another Equate declared anywhere above or below.

The following operators can also be used to declare an Equate:

THIS BYTE

THIS WORD

THIS DWORD

A variable - declared with a DB, DW, or DD directive - has an address and has space reserved at that address for it in the .COM file. But an Equate does not have an address or space reserved for it in the .COM file.

Example:

A - Byte EQU THIS BYTE

DB 10

A_ word EQU THIS WORD

DW 1000

A_ dword EQU THIS DWORD

DD 4294967295

Buffer Size EQU 1024

Buffer DB 1024 DUP (0)

Buffed_ ptr EQU $; actually points to the next byte after the; 1024th byte in buffer.

(6) Extern:

It is used to tell the assembler that the name or label following the directive are I some other assembly module. For example: if you call a procedure which is in program module assembled at a different time from that which contains the CALL instructions ,you must tell the assembler that the procedure is external the assembler will put information in the object code file so that the linker can connect the two module together.

Example:

PROCEDURE -HERE SEGMENT

EXTERN SMART-DIVIDE: FAR ; found in the segment; PROCEDURES-HERE

PROCEDURES-HERE ENDS

(7) GLOBAL: The GLOBAL directive can be used in place of PUBLIC directive .for a name defined in the current assembly module; the GLOBAL directive is used to make the symbol available to the other modules.

Example: GLOBAL DIVISOR:

WORD tells the assembler that DIVISOR is a variable of type of word which is in another assembly module or EXTERN.

(8) SEGMENT:

It is used to indicate the start of a logical segment. It is the name given to the the segment. Example: the code segment is used to indicate to the assembler the start of logical segment.

(9) PROC: (PROCEDURE)

It is used to identify the start of a procedure. It follows a name we give the procedure.

After the procedure the term NEAR and FAR is used to specify the procedure Example: SMART-DIVIDE PROC FAR identifies the start of procedure named SMART-DIVIDE and tells the assembler that the procedure is far.

(10) NAME:

It is used to give a specific name to each assembly module when program consists of several modules.

Example: PC-BOARD used to name an assembly module which contains the instructions for controlling a printed circuit board.

(11) INCLUDE:

It is used to tell the assembler to insert a block of source code from the named file into the current source module. This shortens the source module. An alternative is use of editor block command to cop the file into the current source module.

(12) OFFSET:

It is an operator which tells the assembler to determine the offset or displacement of a named data item from the start of the segment which contains it. It is used to load the offset of a variable into a register so that variable can be accessed with one of the addressed modes. Example: when the assembler read MOV BX.OFFSET PRICES, it will determine the offset of the prices.

(13) GROUP:

It can be used to tell the assembler to group the logical segments named after the directive into one logical group. This allows the contents of all he segments to be accessed from the same group. Example: SMALL-SYSTEM GROUP CODE, DATA, STACK-SEG.